

Sona — Autonomous execution plan

Build instructions for GPT-6 Astra

6 October 2026 · Asia/Seoul · Plan 1.0

IN THIS PLAN

- | | |
|-------------------------------------|--|
| 1. Mission and authority | 6. First complete experiment |
| 2. Fixed product decisions | 7. Requirement coverage |
| 3. Engineering defaults | 8. Release decision and evidence |
| 4. Unattended prerequisite handling | 9. Execution log and delivery contract |
| 5. Build sequence | 10. Scope protection |
-

1. Mission and authority

Build the local production version of Sona defined by [PRD 1.0](#). Complete implementation, verification, packaging, and setup documentation in one autonomous execution. Use the existing design. Do not stop after a prototype, a scaffold, or the first working voice call.

This plan does not start application development. Give the [build prompt](#) to GPT-6 Astra when the build should start.

The PRD controls product scope. This plan defines implementation order and engineering defaults. The user's latest instructions take precedence over both documents.

Execution rules

1. Read repository instructions and the current work before changes. Keep existing user changes.
2. Use the existing checkout. Do not make a worktree unless a repository instruction or the user requests one.
3. Do not ask routine questions or stop for milestone approval. Resolve implementation details with the defaults in this plan.
4. Complete all work that does not depend on an unavailable external prerequisite.
5. Keep a durable execution log. After a context reset, continue from the log and repository state.
6. Repair failed checks and repeat the affected checks. Do not disable tests or substitute simulated success.
7. Do not bypass account permissions, provider approvals, or platform approval controls.
8. Do not publish, deploy publicly, purchase services, or change production calendars. The delivery target is a local application.
9. End with runnable artifacts and a precise verification report. A blocked service test does not justify an unfinished interface or storage layer.

“One execution” means no routine human decisions between stages. It does not mean one code generation without tests or repairs.

What exists now

The repository contains the PRD, interface figures, API inventory, and document tools. It has no application manifest, application runtime, or application test suite. No applicable `AGENTS.md` was found during this inspection. Read instructions again before implementation.

The PRD's API source snapshot is dated 5 October 2026. It contains 354 operations and 227 paths across 31 path families. These counts are inventory facts, not verified service coverage. Reconcile them with current official sources before implementation.

The inspected process did not contain `OPENAI_API_KEY`, `OPENAI_PROJECT_ID`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, or `GOOGLE_CALENDAR_ID`. This is not proof that all authentication routes are unavailable. An official runtime or another configured credential source can hold access.

2. Fixed product decisions

Do not reopen these decisions during the build:

- Three roles: researcher, participant, and maintainer. One person can have several roles. Do not build separate role accounts or a maintainer dashboard.
- Researchers have all agent controls, API parameters, tool settings, transcripts, charts, diagnostics, and exports.
- Researcher view is the default, including when the researcher is the participant.
- Participant view is optional and off by default. A saved study condition can intentionally select it.
- Participant view has essential voice controls, device selection, recording status, and a neutral researcher handoff. It does not show agent answers visually.
- Both views share one call. A view change does not reconnect the call or remove researcher permissions.
- The right panel edits the next-test draft during a call. The applied configuration stays fixed.
- OpenAI supplies AI processing. External services, including Google Calendar, can supply tools.
- Start with a local browser application and server. Use Chrome on Windows and macOS as the initial target.
- Do not retain raw audio by default. Quick-test evidence is temporary. Recorded studies need consent.
- Full public OpenAI API coverage remains part of v1. A voice pilot is an internal milestone, not a substitute for that requirement.
- There is no program-duration deadline or product expiry.

Use these visual targets:

1. Researcher workspace.
2. Participant view.
3. Comparison view.

Keep the dark surfaces, three-column researcher layout, restrained controls, and evidence area. Adapt to smaller laptop viewports without hiding essential call controls.

3. Engineering defaults

Use one TypeScript workspace. Keep the server as one process with internal modules. Do not introduce microservices, a message broker, a cloud database, or another AI-provider integration.

Area	Default
Runtime	Node 24 LTS. Verify dependency compatibility, then pin the exact version.
Package manager	pnpm with an exact version and committed lockfile.
Browser application	React, Vite, and TypeScript. Use semantic HTML and accessible component primitives.
Local server	Fastify. Serve the built browser application from the same origin in production.
Persistence	SQLite with migrations, prepared statements, transactions, and explicit repository functions.
Application schemas	Zod or an equivalent typed validator for Sona-owned contracts.
Provider schemas	A JSON Schema 2020-12 validator for the reconciled OpenAPI 3.1 source. Do not discard schema features through conversion.
AI integration	Official OpenAI SDK and documented Realtime WebRTC transport.
Tool integration	Official MCP client, with an optional adapter to a compatible Codex app-server.
Charts	One maintained React-compatible chart package with accessible data tables.
Tests	Vitest for unit and integration tests. Playwright for browser tests. Separate real-service checks.
Credentials	Server-only environment bindings or an operating-system credential store. Never put an encryption key beside its encrypted secrets.
Release	Built local application, startup commands, migration support, setup guide, and test evidence.

Confirm package maintenance, licenses, and compatibility before installation. Pin the selected versions. If a default is incompatible, select the smallest maintained replacement. Record the reason without asking the user.

Proposed repository layout

apps/web/	researcher workspace, participant view, charts
apps/server/	HTTP routes, sessions, storage, credentials, API runner
packages/agent-core/	portable schema, validation, OpenAI translation, action policy
examples/agent/	minimal runtime using agent-core and local connection bindings
tests/fixtures/	synthetic records and redacted provider fixtures
tests/live/	opt-in tests for actual accounts and devices

docs/setup/	Windows/macOS setup and troubleshooting
docs/verification/	requirement evidence, coverage, known limitations

The example runtime must use the same configuration validator, provider translation, connection interface, and action executor. Do not copy that logic into a second implementation.

Boundaries to define before feature work

- **Configuration:** portable schema version, prompt, model, provider settings, tools, and confirmation policy. Credential values and study UI state are excluded.
- **Session snapshot:** immutable configuration plus study controls, capability revision, application version, runtime versions, and device context.
- **Provider adapter:** capability lookup, configuration translation, call lifecycle, applied-setting evidence, event normalization, and usage records.
- **Connection adapter:** authentication status, tool discovery, schemas, invocation, result reconciliation, and account identity.
- **Action executor:** proposal, approval, dispatch, result, timeout, and unknown-outcome states. An agent cannot grant its own approval.
- **Evidence envelope:** schema version, sequence, session/segment/response/tool identifiers, source, timestamps, clock identity, redacted payload, and completeness.
- **Storage policy:** separate durable snapshots and action journals from temporary quick-test evidence and consented study evidence.

Use interfaces for these boundaries. Implement only the OpenAI adapter. Do not build a generic integration framework beyond the interfaces needed by Sona and the example runtime.

4. Unattended prerequisite handling

Perform this check at the start. Record availability without exposing secret values.

Prerequisite	Autonomous action	If unavailable
OpenAI API access	Reuse configured server credentials. Check project access and selected model availability.	Complete offline implementation and tests. Mark live voice and usage verification blocked.
Existing ChatGPT connections	Use the official runtime's authentication mechanism. Examine tool discovery and account selection.	Try the documented direct MCP route. Do not extract cookies or transfer tokens from ChatGPT.
Calendar authorization	Reuse an authorized account and a dedicated test calendar. Validate scopes and restrictions.	Complete the adapter and OAuth setup flow. Mark service read/write verification blocked.
Test calendar	Use an explicitly designated test resource. Keep attendees and invitations disabled.	Do not select a personal or production calendar as a substitute.
Laptop/browser access	Use provisioned Windows/macOS devices and browser control if available.	Run synthetic browser tests. Keep physical microphone, speaker, and device-switch results unverified.

Prerequisite	Autonomous action	If unavailable
Network access	Use supported network configuration and current official sources.	Continue independent work. Record source or service access failures precisely.
API entitlements	Inspect access per operation and project.	Show the restriction in the API library and coverage report. Do not remove the operation.

A tool connection supplied to the coding assistant is not automatically a connection available to the Sona application. Tests must exercise Sona’s own runtime and authentication route.

Do not ask for secrets in chat. Do not pause to request access during unattended execution. Persist the exact missing prerequisite, complete independent work, and include the remaining action in the final report.

Test resources and costs

Use an isolated test project and calendar when available. Use existing grants only within their scope. External writes are limited to test artifacts created for verification.

Use a default admission budget of USD 10 for unattended billable smoke tests unless the environment provides a different test budget. Before a test, calculate a conservative upper bound from dated official prices. If a bound is unavailable, do not start that paid test.

Limit call duration, concurrency, output, retries, and job size. Include transcription and tool charges in the estimate. An application admission budget is not a provider-enforced invoice cap. Report observed usage and charges that remain unknown.

Do not launch fine-tuning, batch generation, paid media jobs, or destructive administrative requests merely to populate coverage. Use contract tests unless dedicated resources and a sufficient configured budget make the live test appropriate.

5. Build sequence

All packages below belong to one autonomous run. Checkpoints are internal evidence gates, not requests for approval.

Package	Dependency	Runnable result
B0. Foundation and capability baseline	None	Local application, shared contracts, migrations, CI, and source ledger.
B1. Connection feasibility	B0	Authenticated Calendar tool access through a supported route.
B2. Voice and researcher workspace	B0	Real English/Korean voice loop with researcher evidence. Add the tool read when B1 is available.
B3. Configurations and external actions	B1 + B2	Named revisions and a spoken, approved Calendar write with recovery.
B4. Studies, participant view, and persistence	B2 + B3	Consented study runs, optional participant UI, durable evidence, retention, and deletion.

Package	Dependency	Runnable result
B5. Comparisons, exports, and portable runtime	B4	One complete experiment and an exported agent that operates outside Sona's study database.
B6. Full API library	B0 onward	All reconciled public operations represented, editable, validated, and executable where authorized.
B7. Release verification and packaging	B1–B6	Installable local release with evidence and a truthful readiness decision.

Start B1 and the voice portion of B2 as soon as B0 permits them. An OAuth failure must not prevent voice work. Build the B6 schema machinery with B0. Expand it after the voice experiment works.

B0 — Foundation and source reconciliation

Work

1. Inspect instructions, existing files, runtime tools, credentials, and network configuration.
2. Create the minimal workspace, pinned dependencies, development commands, production build, and CI.
3. Bind the server to loopback. Validate Host and Origin. Establish a local session mechanism that rejects unrelated browser origins.
4. Add server-only credential handling and redaction before logs or diagnostics can capture secrets.
5. Define the shared contracts and first SQLite migration. Put application data outside the Git checkout.
6. Reconcile the pinned API inventory with current official reference pages, model guides, SDKs, transport events, and lifecycle notices.
7. Record each parameter's type, permitted values, omission/default semantics, dependencies, model/transport restrictions, and change timing.
8. Save source revisions and hashes. Keep the earlier capability revision for historical configurations.

Gate

A clean checkout installs with the lockfile and starts the local application. A database write survives restart. Invalid requests and foreign origins are rejected. A deliberate error does not expose a secret canary. The capability ledger distinguishes documented, implemented, and verified behavior.

Evidence

Setup instructions, dependency decisions, migration result, CI results, and the reconciled capability ledger. No parameter control may rely only on an image label.

B1 — Connection feasibility and Calendar access

Work

1. Probe a compatible released Codex app-server through documented interfaces. Record the runtime version and account context.
2. Test discovery and direct tool invocation. Prove a Calendar read through Sona's connection interface.
3. Determine whether an existing ChatGPT connection works without a new Google grant. Record reuse and new authorization as different results.

4. If reuse fails, implement direct MCP authorization against the documented Calendar service. Verify audience, scopes, callback requirements, and organization restrictions.
5. Use a dedicated calendar. Inspect actual tool schemas. Enforce selected tools and calendar restrictions in Sona.
6. Exercise one deliberately approved test write through a minimal durable action record. Save the operation and external event identifiers before claiming success.
7. Test revoked access, expiry, reconnection, and account separation. Clean up only test artifacts created by this run.
8. Add a connection library for supported plugin manifests and MCP definitions. Do not execute plugin install scripts or hooks.

Gate

A read and approved write complete on the designated calendar. Wrong-account, unselected-tool, and forbidden-resource calls fail before dispatch. Failures return to the original draft. Reuse is advertised only when the same-account reuse test succeeds.

If a route requires an intermediary AI agent, do not silently add it. Prefer the direct MCP route. If neither route meets the PRD, record the integration blocker and continue independent packages.

Evidence

Redacted requests/results, provider event identifiers, auth-route decision, scope restrictions, and failure results. A manifest URL or tool list is not a successful integration test.

B2 — Real voice loop and the primary workspace

Work

1. Build the final researcher layout early: sidebar, call area, evidence tabs, and right control panel.
2. Connect browser audio through WebRTC using the documented server control path. Keep long-lived credentials on the server.
3. Give Start, Mute, Devices, and End session functional behavior. Do not use cosmetic state changes in place of media operations.
4. Separate connection, microphone, playback, tool, and recording states.
5. Normalize events and expose live transcripts with their sources. Input transcription must be disclosed and configurable.
6. Compare requested and provider-returned configuration. Mark non-returned fields as Sent, not confirmed as applied.
7. Keep quick-test evidence in memory. Persist only its configuration snapshot and necessary redacted action journal.
8. Exercise English and Korean speech. Add a Calendar read as soon as B1 permits it.

Gate

A real laptop microphone reaches OpenAI, spoken output plays, and interruptions stop speech correctly. Mute stops transmission. End session stops capture and new tool dispatch. Permission denial, device loss, playback failure, and reconnect have recovery paths.

Researcher view stays open during self-tests. Draft edits target the next call. A view change or evidence-tab change does not create a second connection.

Evidence

Browser/OS/device versions, actual service identifiers, redacted event traces, and specific input/output/interrupt/mute/stop results. Headless audio fixtures do not satisfy the real-device gate.

B3 — Configuration revisions and controlled actions

Work

1. Implement neutral agent definitions, autosaved drafts, named revisions, duplication, history, and field differences.
2. Share validation between Form, JSON, imports, and runtime execution. Keep omitted values distinct from explicit defaults.
3. Store a fixed snapshot before a measured call. Show the applied configuration separately from the next-test draft.
4. Add tool definitions, instruction versions, schema hashes, account bindings, and confirmation policies.
5. Complete the action executor started in B1. Use durable states for proposal, approval, dispatch, result, failure, and unknown outcome.
6. Bind approval to a proposal identifier, canonical arguments, destination, account, and expiry. A changed proposal needs new approval.
7. Obtain approval from participant speech after completed proposal playback. Use the input source, not the model's statement that approval exists.
8. Use a conservative English/Korean confirmation parser. Reject ambiguity, negation, correction, overlap, expiry, and interrupted proposal playback. Do not use a substring match for "yes."
9. Prevent duplicate approval and duplicate dispatch. On uncertain writes, reconcile before a retry. Bound read retries and record visible attempts.

Gate

A spoken confirmation causes one Calendar write. Silence, rejection, interrupted delivery, changed arguments, stale approval, duplicate callbacks, and account changes do not authorize a write. Killing the process after dispatch does not cause an automatic duplicate on restart.

If reliable voice approval is unavailable, block the write or disclose host-required visual approval as the PRD permits. Do not report a voice-only approval test as passed when a visual step was necessary.

Evidence

A real read/write session, adversarial approval tests, crash/restart tests, tool-schema change tests, and configuration-application records.

B4 — Studies, participant view, and evidence storage

Work

1. Add studies, tasks, fixed conditions, manual order, AB/BA order, measures, consent policy, and pseudonyms.
2. Keep researcher participation separate from account roles and view selection.
3. Implement optional Participant view and neutral handoff. New quick tests and conditions keep it off.

4. Add device selection in both views. Preserve mute during input changes. Stop the old track before the new one transmits.
5. Detect browser output-selection capability. Use System output with operating-system instructions when selection is unavailable.
6. Record view/device changes, linked reconnect segments, deviations, exclusions, and missing evidence.
7. Store consented transcripts, events, notes, ratings, outcomes, and annotation history. Keep participant and generated transcript sources separate.
8. Implement the default 30-day retention policy, configurable retention, deletion, startup cleanup, and periodic cleanup.
9. Handle out-of-order and duplicate events, partial/final transcripts, late tool results, failed storage, and interrupted sessions.

Gate

A researcher can complete a study as its participant with all research evidence visible. A supervised participant sees only essential controls and spoken agent answers. Handoff and reload do not reveal private panels. Capture does not start without applicable consent.

A saved session survives restart with its original snapshot. Snapshot failure prevents the measured run. A view change preserves the session ID. An expired record is removed when cleanup runs. Quick-test evidence does not enter a later study record.

Evidence

Browser workflow results, persistence/recovery tests, consent and deletion tests, and both view paths on the supported laptop environments.

B5 — Analysis, exports, and agent portability

Work

1. Implement event timelines and the response/tool timing definitions from PRD section 8. Use a common monotonic clock for each measured interval.
2. When a required event or common clock is unavailable, show a missing metric. Do not substitute a different latency under the same name.
3. Add configurable ratings, outcomes, notes, coded errors, and recovery events.
4. Build descriptive comparison graphs and accessible tables. Show participants, sessions, observations, exclusions, and missing counts separately.
5. Filter by condition, view, and researcher participation. Keep unlike view/participation groups separate by default.
6. Export versioned JSON and CSV with joins, units, definitions, source versions, and missing-data reasons. Protect against spreadsheet formula execution.
7. Export `sona-agent.json` without participant data or credentials.
8. Build the example runtime with the same agent-core code and a new authorized connection binding.

Gate

Complete the end-to-end experiment in section 6. Displayed results agree with exported records. Missing values do not become zero. The exported agent operates without Sona's study database and preserves the same confirmation policy.

Evidence

A redacted evidence bundle, export round-trip results, malformed-import tests, spreadsheet-safety tests, and an independent-runtime read/write result.

B6 — Full OpenAI API library

Work

1. Expand the reconciled capability ledger into searchable operation and parameter views.
2. Handle nested objects, arrays, unions, enums, references, files, binary results, null values, and omitted values.
3. Validate path, query, header, and body inputs. Keep auth fields as server-side credential references.
4. Implement execution handlers for request/response, streaming, bidirectional sessions, multipart uploads, pagination, asynchronous jobs, cancellation, and callbacks where documented.
5. Use a separate lifecycle for generic API runs. An API-library request must not mutate a measured voice session behind its executor.
6. Supply local handling for callback routes where feasible. Explain the public callback URL requirement when the provider must reach the server.
7. Track catalog, editing, validation, execution, account restrictions, lifecycle, and live verification separately for each operation.
8. Show restrictions and missing entitlements in the interface. Do not hide gaps or invent a default.
9. Add explicit intent checks for destructive and billable administrative operations. Catalog loading must never launch a paid job.
10. Exercise all execution shapes with contract fixtures. Use authorized live checks across the available service families.

Gate

Every currently offered operation in the reconciled inventory has an implementation disposition. All applicable fields can be represented without loss, validated, and sent through the correct handler. Any missing handler is incomplete implementation, not an account restriction.

Provider-restricted live tests can remain explicitly unverified. They must not appear as passed. Full source coverage does not mean that every endpoint was exercised against a live account.

Evidence

An operation-level coverage report with source references, handler, schema tests, account restrictions, live test IDs, and gaps. Recompute the denominator from current sources. Do not use 354 as a fixed completion target.

B7 — Production verification and delivery

Work

1. Repair integration failures across B1–B6. Remove dead controls, placeholder responses, and production fallbacks to mock data.
2. Run type checks, builds, unit tests, integration tests, and browser tests. Confirm that the expected tests actually executed.
3. Test the final layouts against all three visual targets at 1440 × 1024 and 1280 × 800.
4. Test keyboard use, input focus, screen readers, contrast, reduced motion, 200% zoom, and reflow against WCAG 2.2 AA.
5. Inspect built assets, logs, diagnostics, exports, URLs, and source maps for secret canaries. Check Origin/Host enforcement, OAuth state, callback handling, and connection allowlists.
6. Test clean installation, startup, shutdown, restart, schema migration, backup, and recovery on Windows and macOS.
7. Keep raw study data out of source control. Keep sanitized verification evidence separate from private test records.
8. Verify the portable runtime with an independent account binding. Do not describe an isolated test account as a completed teammate usability test.
9. Write setup, maintenance, credential, connection, cost, troubleshooting, and release documentation.
10. Build a release archive with source/version identity and checksums. Keep publication local unless separately authorized.

Gate

Apply the readiness decision in section 8. Fix any code-caused failure before ending. If an external prerequisite prevents a mandatory test, deliver a release candidate with that gate blocked.

6. First complete experiment

Use the optional Calendar example only as a validation fixture. Sona must still permit other agents and tasks.

1. Add a neutral demonstration agent through the UI. Connect the dedicated test calendar.
2. Duplicate its condition. Use two officially supported speed values after capability verification. The existing example suggests 0.85 and 1.15.
3. Keep prompt, model, voice, language, devices, tools, and confirmation policy fixed.
4. Use a researcher participant with Researcher view. Obtain consent and complete A/B runs in a recorded order.
5. Read calendar availability, propose an appointment, obtain spoken approval, and create the event once.
6. Include a rejection, an interrupted proposal, and a service failure. Preserve the true outcomes.
7. Repeat the voice and approval checks in Korean with a separate language condition.
8. Exercise Participant view in a separate planned condition. Do not mix the two views into a clean comparison.
9. Add ratings and coded outcomes. Compare conditions with counts and missing values visible.
10. Export JSON, CSV, and the agent configuration. Operate the exported agent through the example runtime.

This is a functional test, not a claim of statistical significance or an optimal configuration.

7. Requirement coverage

Each identifier needs implementation evidence and a verification disposition.

PRD requirement	Build packages	Required evidence
AG-01	B2, B3	Add a second agent without application-code changes.
CF-01	B3, B4	Immutable revisions, duplication, history, and field differences.
CF-02	B0, B2, B3	Schema validation and requested/returned/observed records.
AP-01	B0, B6	Reconciled operation inventory with no unexplained omissions.
AP-02	B3, B6	Lossless Form/JSON editing and parameter validation.
AP-03	B6	All necessary execution shapes with truthful live-test status.
CN-00	B1	Existing-account reuse result or documented failure and fallback.
CN-01	B1, B3	Definition import, authorization, discovery, and tool restrictions.
CN-02	B1, B3	Source/runtime versions, schema hashes, and change handling.
VS-01	B2, B7	Actual English/Korean microphone and speaker sessions.
VS-02	B2, B4	Interruption, mute, end, connection loss, and recovery.
TL-01	B1, B3	Actual reads, approved writes, and rejected writes.
TL-02	B3	Duplicate prevention and unknown-result reconciliation.
EX-01	B4, B5	Tasks, conditions, ordering, notes, and ratings.
EV-01	B2, B4, B5	Snapshots, transcript sources, events, clocks, and gaps.
EV-02	B5	Descriptive comparisons and linked JSON/CSV exports.
PT-01	B3, B5, B7	Exported agent operates without the study database.
DH-01	B0, B4, B5	Consent, redaction, retention, deletion, and safe exports.
DX-01	B0, B7	Documented installation and an independent teammate setup result.

PRD requirement	Build packages	Required evidence
UX-01	B2, B4, B7	Saved context and measured interaction targets.
UX-02	B3, B4	Optional Participant view and verified confirmation behavior.
UX-03	B2, B4, B7	Laptop layouts, accessibility, and failure recovery.
UX-04	B2, B4	Researcher controls and evidence during researcher participation.
UX-05	B4, B5	Device/view controls, event records, and protocol deviations.

8. Release decision and evidence

Keep implementation status separate from test status. A blocked or skipped test is not a pass.

Production-ready local v1

Use this status only when all mandatory conditions below are satisfied:

- The full PRD feature set is implemented, including the API library and portable runtime.
- Core voice, Calendar reads/writes, approval, recovery, persistence, and export have successful real-service results.
- The claimed Windows/macOS environments have successful real-device evidence.
- Consent, credentials, local-server access, redaction, retention, and deletion checks pass.
- Researcher participation and optional Participant view behave as specified.
- Build, migration, installation, and accessibility checks have no unresolved release-blocking failure.
- Independent teammate setup, configuration portability, and the PRD's researcher usability checks have evidence. Automated clean-install tests do not substitute for a teammate or participant.
- API coverage gaps are classified. Account-restricted verification is visible, and missing implementation is not concealed.

Do not claim that all OpenAI APIs are live-tested unless the operation records prove it. Do not claim that the application removes third-party service limits.

Release candidate — verification blocked

Use this status when implementation is complete but a mandatory external test lacks credentials, permissions, a device, or independent teammate access. Deliver all code and executable tests. Identify the missing prerequisite and the exact test command.

A missing prerequisite must not become a reason to stop unrelated work. No user response is needed to complete the release candidate.

Incomplete

Use this status if required code or a handler remains missing, or a code-caused test failure remains unresolved. Do not rename missing implementation as a verification blocker.

Evidence record

For each test, save its requirement IDs, application revision, capability revision, environment, account class, timestamp, command, result, and sanitized artifact path. Keep actual secrets and participant records out of these artifacts.

For live tests, include provider request/session IDs and test-resource IDs after appropriate redaction. For device tests, include OS, browser, microphone/output route, language, and the observed result.

9. Execution log and delivery contract

Maintain `docs/implementation/execution-log.md` during the build. Record the active package, completed work, decisions, test results, failures, external blockers, and the next useful action.

The log must permit another Astra turn to continue without asking the user to reconstruct context. Record files and revisions, not only prose claims.

Provide these commands with their documented meanings. They are proposed command names until B0 implements them:

Command	Meaning
<code>pnpm install --frozen-lockfile</code>	Reproduce dependency installation.
<code>pnpm dev</code>	Start browser and server development processes.
<code>pnpm build</code>	Build the local production application.
<code>pnpm start</code>	Start the built application and apply supported startup migrations.
<code>pnpm check</code>	Type checks and static code checks.
<code>pnpm test</code>	Unit and integration tests with a test-count report.
<code>pnpm test:e2e</code>	Browser workflows with synthetic fixtures clearly identified.
<code>pnpm test:live</code>	Opt-in service/device checks with prerequisite and budget checks.
<code>pnpm coverage:api</code>	Operation and parameter implementation/verification report.
<code>pnpm example:agent</code>	Operate a supplied portable agent configuration.
<code>pnpm package</code>	Make the local release archive and checksums.

Do not call a proposed command verified until it exists and succeeds on the reported revision.

Deliver the application, example runtime, migrations, tests, setup guide, API coverage report, requirement evidence, and release archive. Document normal first-use authentication. A new teammate must supply their own authorized credentials.

The final response must contain the readiness status, startup instructions, passed verification, remaining restrictions, and artifact paths. It must not end with a routine permission question or an offer to do work that remains authorized.

10. Scope protection

Use the existing design instead of another design-selection cycle. Use the official ASD-STE100 specification for editorial work. Do not run the archived Python language checker or replace it with another automatic compliance score.

Do not reduce full v1 to the early voice milestone. Do not add another AI provider, a remote observer console, a maintainer dashboard, or shared hosting to compensate for an integration blocker.

If a provider capability is unavailable, represent that fact accurately. Implement the documented fallback when it meets the PRD. Otherwise, record the unresolved capability and continue all other work.